マルウェア動的解析における観測性向上を目的としたAPIフック自動生成手法の提案

高林 りずむ[1,a)]　グエン チ ヴァン アン[2]　田辺 瑠偉[2]　吉岡 克成[3,2]

概要：マルウェア動的解析時に実行した検体が呼び出した API の動作を記録する技術（API フック）は，マルウェア検体の挙動を把握する上で重要である．しかし，フックされていない API についてはその動作を観測できないという課題があり，特に解析環境を検知してその挙動を変化させる回避型マルウェアとの間で激しい攻防が続いている．本研究では，大規模言語モデル（LLM）を用いて未実装の API フックを自動生成する手法を提案する．提案手法は，解析レポートから回避型マルウェア検体が呼び出す API を特定し，フックされていない場合において，API 仕様情報とあらかじめ LLM により導出したフック生成規則に基づいて当該 API をフックするスクリプトを生成する．さらに，生成した API フックを解析システムに組み込んで API ログを取得するとともに，ビルドやログ出力に関するエラーが発生した場合には LLM を用いて自動修正する．代表的なオープンソースのマルウェア動的解析システムである CAPEv2 を用いた調査では，1,520 個の解析回避実証コードが呼び出す 531 種の Windows API のうち，424 種（79.9%）がデフォルトのフック群では観測できないことが明らかとなった．そこで，既存のサンドボックス検知ツールを元に作成した回避 PoC 163 検体に提案手法を適用して 222 件のフックを生成した．この結果，評価対象の 136 検体では，回避 PoC のソースコードが明示的に呼び出す API は検体ごとに重複を除いて 461 件だった．このうち 411 件について解析レポートで記録でき，提案手法適用前後で解析レポートに記録される割合が 58 ポイント（267 件）向上した．また，生成したフックが記録した引数の値を実測と照合したところ，96.0%が一致した．提案手法は，マルウェア動的解析システムの観測性能向上に資するものであり，回避型マルウェア検体以外にも有効利用できる可能性がある．

A Proposal of Automatic API Hook Generation Method for Improving Behavioral Visibility in Malware Dynamic Analysis

Rizumu Takabayashi[1,a)]　Nguyen Thi Van Anh[2]　Rui Tanabe[2]　Katsunari Yoshioka[3,2]

Abstract: Dynamic malware analysis executes a sample in a sandbox and observes its behavior by recording Windows API calls through API hooking. CAPEv2, a representative open-source sandbox, provides hooks for Windows APIs chosen to capture the behavior of malware in general. Evasive malware, however, which detects the analysis environment and changes its behavior, may use APIs that CAPEv2 does not hook, leaving the analyst unable to fully observe such behavior. In a preliminary study, we examined the 531 Windows APIs invoked by 1,520 sandbox-evasion proof-of-concept (PoC) programs obtained from prior work, and found that 424 (79.9%) of them cannot be observed by default. Existing tools only generate hook skeletons, and completing an API hook with logging functionality still requires manual work. In this paper, we propose a system that automatically generates API hooks for CAPEv2 in order to improve the observability of malware behavior. The proposed system analyzes the analysis report to identify the APIs invoked by a sample for which no hook exists, and automatically generates API hooks with logging functionality based on the API specification and on hook generation rules derived in advance by an LLM. It then integrates the generated hooks into the sandbox and verifies that logs are obtained, having an LLM repair any hook that causes a build or logging error. Applying the system to 163 of the 1,520 PoCs, we generated 222 hooks. On the 136 samples used for evaluation, the APIs explicitly invoked in the source code amount to 461 when counted per sample with duplicates removed, of which 411 were recorded in the analysis report; the proportion recorded rose by 58 points (267 APIs). Comparing the argument values logged by the generated hooks against the values observed at run time, 96.0% of them matched.

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

マルウェアによる被害は増加の一途を辿っており，解析環境（以降では，サンドボックス）内でマルウェア検体を実際に実行してその挙動を把握する動的解析は，マルウェア対策を考える上で必要不可欠な技術となっている．Windows 上で動作するマルウェアの多くは API を呼び出して機能を実現するため，API をフックして呼び出しを記録することが解析の中心となる．オープンソースのマルウェア動的解析システムである CAPEv2[1] にもこの機能が備わっており，多くの解析者に利用されている．

一方で，サンドボックスを検知してマルウェア本来の挙動を隠蔽することで解析を回避するマルウェア（以降では，回避型マルウェア）が大きな問題となっている [2]．その多くは実行環境の情報を取得してサンドボックスであるかを判断するが，CAPEv2 をはじめとする既存のサンドボックスが，こうした API をどの程度フックできているかを調査した研究は我々の知る限り存在しない．

API フックの実装には，Windows システムの内部構造に関する専門知識が必要であり一般的に技術的なコストが高い．そのため，実装を支援するツールとして cape-hook-generator[3] が存在するが，生成されるのはシグネチャから得られる雛形にとどまり，解析レポートに API の呼び出し結果を出力するには追加の作業を要する．

そこで，本研究では回避型マルウェアを主な対象として，これらのマルウェアが呼び出す API フックの生成，解析システムへの組み込み，およびログ取得の検証を自動化する手法を提案する．マルウェア検体を実行後，(1) 解析レポートを分析して検体が呼び出す API のうち未実装の API を特定して，(2) API 仕様情報とあらかじめ LLM により導出したフック生成規則に基づいて，解析レポートへのログ出力機能を備えた API フックを自動生成し，(3) 生成したフックを解析システムに組み込んでログが記録できることを検証する．なお，API フックのビルドやログ出力にエラーが生じた場合には，LLM を用いて修正する．

事前調査としてインストール直後の CAPEv2 環境を用意し，文献 [4] で公開している解析回避実証コード（以降では，回避 PoC）1,520 件を調査した．各ソースコードから，Windows SDK のヘッダに宣言を持つ関数の呼び出しを Clang[5] の抽象構文木を用いて抽出し，Windows API 531 種を得た．続いて，各ソースコードをコンパイルして CAPEv2 で実行し，capemon が既定で設置するフック 586 種と突き合わせたところ，424 種（79.9%）はデフォルトのフック群では観測できないことが明らかとなった．これには，回避型マルウェアがプリンタの有無を調べる API 呼び出し（EnumPrintersW）などが含まれる．

このデータセットは，実装されたベース検体群と，これらをもとに LLM により多様化された検体群からなる．ベース検体群のうち，サンドボックス検知ツールである Pafish[6] および Al-Khaser[7] を元とする解析回避コード 64 個，Galloro ら [2] が体系化した回避技術の分類に基づく解析回避コード 99 個をコンパイルして回避 PoC 163 検体を作成した．これらの検体を抽出構文木を用いて解析したところ 306 種がフック生成対象であり，提案手法を用いて 222 種についてフックを生成して capemon へ組み込むことに成功した．

さらに，これらの検体を CAPEv2 で解析して実行が完了しなかった検体を除いた 136 検体を用いて，提案手法適用前後の解析レポートの API コールログを比較した．評価対象を定めるため，ソースコードから特定した API について，検体自身が出力する実行ログを用いて実行経路を追跡した．評価対象とする API の件数は，検体ごとに重複を除いて種類を数えたところ 549 件となり，実行を確認できた 461 件とする．提案手法を適用した結果，解析レポートに記録された API は，提案手法適用前は 144 件（31.2%）であったのに対し，提案手法適用後は 411 件（89.2%）へと増加しており，観測性が高まったことが確認された．

生成したフックが引数の値を正しく記録するかを検証するため，各 API についてその API を呼び出す検証プログラムを用意し，API 呼び出しの直後にプログラム自身が渡した値を解析レポートに記録させた．これらを生成したフックが記録した値とを引数ごとに照合したところ，照合できた 667 引数のうち 640 引数（96.0%）で値が一致し，フック生成規則が高い精度で機能することが確認された．

2. 関連研究

2.1 回避型マルウェアの実態

回避型マルウェアは，実行環境が解析環境であることを検知すると悪性な挙動を隠蔽し，動的解析を妨害する．Galloro ら [2] は，既知の回避技術 92 種類を収集して 16 のカテゴリに分類し，10 年間にわたって収集した 45,375 検体を対象に，各回避技術がどの程度用いられているかを調査した．その結果，回避挙動を実装した検体の割合は 2010 年の 67.8%から直近の年には約 80%へと増加しており，回避が継続的な脅威であることが示された．

2.2 サンドボックスの回避耐性の向上

回避型マルウェアへの対処として，サンドボックスを検

[1] 横浜国立大学大学院環境情報学府
Graduate School of Environment and Information Sciences, Yokohama National University
[2] 横浜国立大学先端科学高等研究院
Institute of Advanced Sciences, Yokohama National University
[3] 横浜国立大学大学院環境情報研究院
Faculty of Environment and Information Sciences, Yokohama National University
[a)] takabayashi-rizumu-mt@ynu.jp

© 2026 Information Processing Society of Japan

知されにくくする研究がある．伊藤ら [8] は，回避技術の実証コードを LLM に分析させ，ゲスト OS，仮想化基盤，サンドボックス本体のカスタマイズ方法と適用手順を生成する手法を提案した．CAPE サンドボックスを対象とした評価では，43 検体中 31 検体で解析に成功している．

こうした回避耐性の評価には，多様な回避技術を含む検体群が必要となる．松澤ら [4] は，既存の評価ツールに含まれる実証コードを LLM により統一的な形式で再実装するとともに，回避技術の分類に基づいて新たな実装を生成し，4,508 件の候補から検証を通過した 1,520 件をデータセットとして公開しており，本研究にも活用する．

本研究は，回避が行われた場合にその挙動を解析レポート上に残すことを目指す点で先行研究と目的が異なり，回避を防ぐ手法とは補完的な関係にある．

2.3 API フックの拡張

サンドボックスが記録できる情報そのものを増やす研究もある．Raducu ら [9] は，CAPE の解析レポートから機械学習に利用可能な実行トレースデータセットを生成するフレームワーク MALVADA を提案し，既存の実行トレースデータセットが API のパラメータや戻り値といった文脈情報を欠いている点を課題として挙げている．これは本研究の問題意識と一致する．同フレームワークではこの情報を得るために capemon を改造してフック対象の API を追加しており，その作業を補助するツールとして cape-hook-generator[3] を併せて公開している．

しかし，cape-hook-generator が出力するのは雛形であり，フック対象 API の選定，capemon への統合，再コンパイルはいずれも人手に委ねられている．本研究は，これらを未フック API の特定からログ取得の検証まで一貫して自動化する点で先行研究とは異なる．

3. 関連技術

3.1 CAPEv2 と capemon の API フックの構成

オープンソースの動的解析システムの一種である CAPEv2[1] は，マルウェア検体を隔離された環境で実行してその挙動を記録する機能を有しており，多くの解析者の間で利用されている．解析を制御するホストと検体を実行するゲストの仮想マシンから構成され，ゲスト内に用意されたエージェントが検体のプロセスに監視用の DLL である capemon[10] を注入する．その後，capemon は対象とする API の入口にフックを設置し，呼び出しを観測してホストへ送信する．ホストは受け取った記録を集約し，API 呼び出しの列，生成されたプロセス，アクセスされたファイルやレジストリなどを含む解析レポートとして出力する．したがって，検体のどの API 呼び出しが解析レポートに現れるかは，capemon がどの API にフックを設置しているかによって決まる．しかし，capemon がフックする API は，一般的なマルウェアの挙動を捉える目的であらかじめ選定されたものであり，回避のために用いられる API がその範囲に含まれるとは限らない．

capemon のフックは対象の API ごとに 1 つの関数として C 言語で実装される．その形を**図 1** の下部に示す．フックは `HOOKDEF` マクロで定義され，`Old_`を接頭辞とする関数ポインタを介して本来の API を呼び出し，その戻り値を呼び出し元へ返す．この構造により，API の動作を変えずに呼び出しを観測できる．フックを 1 つ追加するには (a) フック本体，(b) そのプロトタイプ宣言，および (c) 対象の API をどの DLL で捕捉するかを指定する登録行 `HOOK(dll, api)` の 3 つの断片が必要になる．capemon は実装をファイル操作，レジストリ操作，ネットワーク通信などのカテゴリごとのソースファイルに分けており，本体はいずれかに配置する必要がある．これらはいずれも，API のシグネチャ，すなわち戻り値の型，呼び出し規約，および各引数の SAL 注釈・型・名前から機械的に定まる．図 1 上部の入力ブロックがこれにあたり，**表 1** では転記に分類される．なお図中で色を付けた箇所は，API ごとに値が変わる部分であり，以降これをプレースホルダと呼ぶ．

3.2 cape-hook-generator

cape-hook-generator[3] は, capemon へ新たなフックを追加する作業を支援するツールである．同ツールは, Windows API のシグネチャを収録した `winapi_categories.json`（以降，シグネチャ DB）を参照する．シグネチャ DB は API ごとに，属するカテゴリ，戻り値の型，呼び出し規約，および各引数の SAL 注釈・型・名前・説明文を持つ．

同ツールは，ここから対象 API のシグネチャを取得し，フック本体，プロトタイプ，および登録行の 3 つを出力する．ただし出力される本体は本来の API を呼び出すだけであり，図 1 の転記のプレースホルダのみが埋まる．解析レポートへログを出力する処理を持たないため，このフックを組み込んでも解析レポートには何も記録されない．

3.3 LOQ による記録

フックが観測した内容を解析レポートへ残すには，フック本体の中でログ出力を行う構造を持つ必要がある．capemon はこのためのマクロ LOQ を備えている [10]．図 1 の `LOQ_`行がこれにあたる．以下に述べるバリアントと書式指定子はいずれも，capemon が実装しているものである．

バリアント（`VARIANT`）は, その API 呼び出しを成功とみなす条件を指定する．capemon は `LOQ_ntstatus`, `LOQ_bool`, `LOQ_handle` などを定義しており，たとえば `LOQ_ntstatus` は `NT_SUCCESS` が真であることを成功条件とする．

書式（`FORMAT`）は 1 文字の指定子を並べた文字列である．指定子 1 文字につきレポート上の表示名（`LABEL`）を 1 つと，記録する対象を与える式（`EXPR`）をその指定子が

© 2026 Information Processing Society of Japan

```
/* ---- 入力: API シグネチャ---------
 * RETURN_TYPE CALLCONV API_NAME(
 * SAL_1 PARAM_TYPE_1 PARAM_NAME_1,
 * ...
 * SAL_N PARAM_TYPE_N PARAM_NAME_N
 * );
 * ------------------------------- */
HOOKDEF(RETURN_TYPE, CALLCONV, API_NAME,
    SAL_1 PARAM_TYPE_1 PARAM_NAME_1,
    /* ... i = 1..、シグネチャと同じ順序 N ... */
    SAL_N PARAM_TYPE_N PARAM_NAME_N
) {
    RETURN_TYPE ret = Old_API_NAME(
        PARAM_NAME_1, ..., PARAM_NAME_N);
    LOQ_VARIANT("CATEGORY", "FORMAT",
        "LABEL_1", EXPR_1,
        /* ... j = 1..、MFORMAT と同じ順序... */
        "LABEL_M", EXPR_M);
    return ret;
}
```

図 1 capemon の API フックの構造．本研究では，上部の API シグネチャを入力とし下部のフック本体を生成する．フックの生成とは，色を付けたプレースホルダの値を決めることに帰着する（由来は表 1）．引数 N 個のうちログに出力するのは M 個であり，一般に $M \neq N$ である．

表 1 フックの構造におけるプレースホルダの由来

由来	プレースホルダ	本研究での扱い
転記	RETURN_TYPE, CALLCONV, API_NAME, SAL_i, $\mathrm{PARAM_TYPE}_i$, $\mathrm{PARAM_NAME}_i$	既存ツールが転記する
導出	VARIANT, FORMAT	規則で決定する
手動	CATEGORY, LABEL_j, EXPR_j	規則で決定し，決まらない引数は LLM が判断する

要求する個数だけ取る．指定子は引数をどの粒度で記録するかを決める．同じポインタでも p はアドレスのみを，b はバッファの中身を記録するため，引数ごとの選択が解析レポートに残る情報量を左右する．したがって，引数ごとに適切な指定子を選べるかどうかが，解析レポートに有用な情報を残せるかを直接左右する．capemon が実装している指定子の一覧を付録 A.1 に示す．

LOQ_行を構成する VARIANT, CATEGORY, FORMAT, LABEL, EXPR は API のシグネチャに現れない．capemon が定めているのは指定子から記録処理への対応だけであり，API の引数から指定子への対応は与えられていない．これを規則として定めることが本研究の中核であり，4.2 節で述べる．

4. 提案手法

本研究の提案は 2 層からなる．1 つは，3.2 節で述べた cape-hook-generator の出力にログ出力処理を補い，解析レポートに記録されるフックへ変換する生成器 LoggingAPIHookGen である．もう 1 つは，これを組み込み，未フック API の特定からログ取得の検証までを 1 検体ごとに繰り返すシステムである．以下，4.1 節でシステム全体の流れを示したうえで，4.2 節以降で LoggingAPIHookGen が用いる規則と処理を述べる．

4.1 システムの概要

提案システムの全体像を**図 2** に示す．1 検体あたりの処理は 7 つの段階からなり，段階の間で生じる判断を LLM が担う．処理を終えると，その検体の API 呼び出しをより多く記録した解析レポートと，検証を通してフック DB へ蓄積されたフックが得られる．

(1) 初期状態での解析：capemon をインストール直後の CAPEv2 と同じ状態，すなわち本システムが生成したフックを一切含まない状態に戻したうえで，検体を解析する．前の検体のフックを持ち越さず，検体ごとの測定を独立させるためである．あわせて，検体が完走したことを示すために自ら出力する文字列（終了マーカー）が解析レポートに現れたかを記録する．本システムの適用には，このような出力を行う検体が必要となる．

(2) 未フック API の特定：解析レポートのインポートテーブルおよび動的に解決された API の記録から検体が使用する API を抽出し，デフォルトでフックされている API と除外対象の API を差し引いて，フックを実装すべき API を決定する．ただし，capemon にフックが登録されていてもログ出力を持たない API は実装の対象に含める．この場合はフック本体を生成せず，ログ出力処理のみを既存のフックへ追加する．除外対象の 29 種は，当初 LLM が推論により定めた 143 種を実測に基づいて絞り込んだものである（付録 A.5）．2 種はフックを追加すると検体の実行が止まることを実測で確かめたものであり，残る 27 種は雛形を生成できないかビルドが通らないもので，API の性質ではなく本システムの限界による．

(3) フックの用意：対象の API がフック DB に登録済みであれば，そのフックを復元する．未登録の場合は新たに生成する．生成は 2 段からなる．まず既存ツール cape-hook-generator[3] が，シグネチャ DB（3.2 節）を参照して API の雛形を出力する．次に本研究で実装した LoggingAPIHookGen が，雛形にログ出力処理を組み立てて挿入し，解析レポートに記録されるフックへ変換する (4.2 節)．

(4) 組み込み・ビルド・配置：用意したフックを capemon に組み込み，ビルドして CAPE サーバへ配置する．当該検体のフックのみを追加するため，(1) との差分は追加したフックに帰属する．

(5) 検証：同一の検体を同一の環境で再度解析し，解析レポート上で終了マーカーを観測できればログの取得に成功したとみなす．

(6) フック API のパラメータログの可読性のチェック：

© 2026 Information Processing Society of Japan

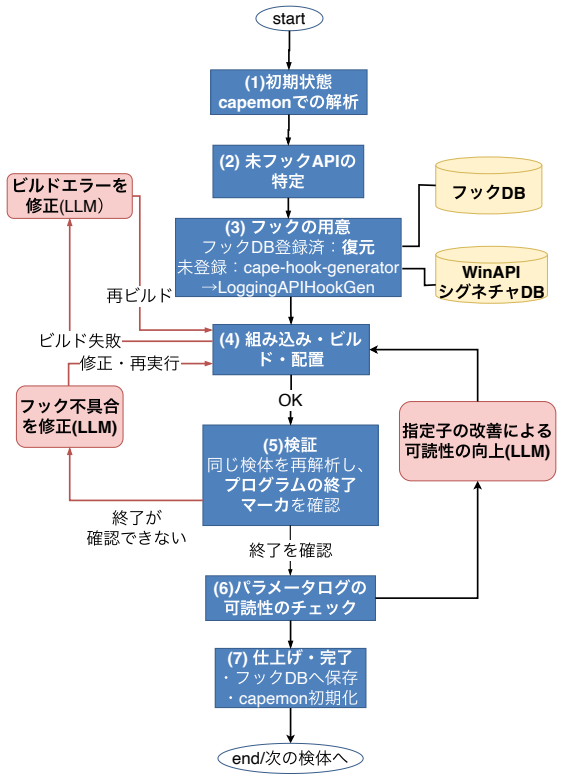


図 2　提案システムの全体像

4.2 節の規則で指定子が決まらず，アドレスのみの記録に留まった引数を対象に，指す先の内容を記録する指定子へ置き換えられるかを確認する．

(7) 仕上げ：検証を通ったフックをフック DB へ登録し，capemon を初期状態へ戻して次の検体へ移る．

この流れには 3 つの反復ループがある．第一に，(4) でビルドが失敗した場合，LLM がコンパイルエラーから原因となるフックを特定し，修正するか実装対象から除外する．エラーが解消するまで繰り返す．第二に，(5) で終了マーカーを観測できない場合，(1) で記録した結果と照合したうえで，原因となるフックを二分探索により特定し，修正または除外する（A.8 節）．ログが取得できるまで繰り返す．第三に，(6) で内容を読み取れない引数があれば，LLM が指定子を調整したうえで (4) へ戻る．

フック DB には検証を通ったフックのみを登録するため，動作しないフックが混入せず，検体を処理するほど生成の対象は減っていく．

LLM の役割は，処理に先立って定める 4.2 節の生成規則と，処理の途中で行う上記 3 つの判断に分かれる．いずれも Anthropic 社の Claude Opus 5 を用い，Claude Code 上で reasoning effort を high に設定して実行する．

4.2　フック生成規則

LoggingAPIHookGen は，既存ツールが埋めずに残す `LOQ_`行のプレースホルダを，API のシグネチャから決定する規則に従って埋める．規則は，戻り値に対するバリアントの選択（`VARIANT`）と，各引数に対する指定子の選択（`FORMAT`）の 2 つからなる．

表 2　指定子の判定規則（上から順に評価）

	引数の型・条件	指定子
1	Unicode 文字列	`u`（ファイル名なら `F`）
2	ANSI 文字列	`s`（ファイル名なら `f`）
3	capemon が展開可能な構造体ポインタ	`O`, `o`, `n`
4	ハンドル	`p`
5	ハンドルへのポインタ	`P`
6	生バッファ	`b`（長さと対にできた場合） `p`（できない場合）
7	整数へのポインタ	`I`（64bit は `X`）
8	整数	`i`
9	上記以外	`p` または `i`

4.2.1　規則の方針

規則の設計には 2 つの方針を置いた．第一に，どの引数も記録から落とさない．解釈できる型のみを記録する方式では，構造体やバッファが解析レポートに現れないため，いずれの条件にも該当しない引数も `p` または `i` で必ず記録する．アドレスのみであっても，その引数が渡された事実は残る．第二に，ポインタの参照は capemon に任せる．出力引数をフック側で読むと，API が失敗して何も書き込まれていない場合に解析対象のプロセスごと停止する．そこで，capemon の `I`，`P`，`X` などは読み取りを例外処理で囲むため安全であり，出力引数に必ず割り当てる（付録 A.1）．

4.2.2　バリアントの選択

バリアントは戻り値の型から決定する．`NTSTATUS` には `LOQ_ntstatus` を，`BOOL` には `LOQ_bool` を，`HANDLE` には `LOQ_handle` を，ポインタ型には `LOQ_nonnull` を割り当てる．整数型については，戻り値 0 を失敗とみなす API が多数派であることから `LOQ_nonzero` を既定とするが，`RegOpenKeyEx` のように 0 を成功とする API も存在するため，一意には決まらない．そこで整数型に対しては既定値を与えたうえで，確認を要する旨をフックに併記する．

4.2.3　指定子の選択

指定子の選択には 2 つの困難がある．第一に，Windows SDK は同一の対象を指す型の別名を多数持つ．そこで `LPCWSTR`，`LPWSTR`，`PWSTR`，`BSTR` のように同一の指定子で扱える型名をあらかじめ集合にまとめ，別名の違いを吸収する．第二に，型だけでは決まらない区別が存在する．文字列を絶対パスとして記録すべきか，および生バッファの長さがどの引数にあるかは型に現れないため，引数名に対する正規表現を併用する．

表 2 に判定の優先順位を示す．規則は上から順に評価し，最初に該当した条件で指定子を確定する．文字列型を最上位に置いているのは，`LPWSTR` が下位の「ポインタ」の条件にも該当してしまうためであり，より限定的な条件を先に置くという原則で並んでいる．

規則 6 の生バッファは，長さを与える引数と対にできた

© 2026 Information Processing Society of Japan

場合に限り b で中身を記録し，できない場合は p に留める（付録 A.2）．規則 9 に落ちてアドレスのみの記録に留まった引数は，規則の適用対象外として印を付け，4.1 節 (6) において LLM が個別に判断する．該当する引数は 3 種に分類でき，うち出力バッファは，書き込まれた長さが戻り値で返されるという事実がシグネチャに現れないため，規則で決定することが原理的に不可能である（付録 A.3）．

4.3 カテゴリと登録先の決定

生成したフックが capemon の中で正しく機能するには，LOQ_行の内容に加えて API のカテゴリと登録先の DLL を決める必要がある．いずれも 4.1 節 (3) において LoggingAPIHookGen が決定する．カテゴリを決めることは，3 節で述べたとおりフック本体を記述するソースファイルを決めることであり，同じ値が LOQ_行の CATEGORY にも入る．

登録される DLL については，同一の API が複数の DLL からエクスポートまたはフォワードされる場合，登録先を誤るとフックが設置されない．たとえば kernel32.dll に登録しても，検体が kernelbase.dll を経由して呼び出していればフックは効かない．そこで LoggingAPIHookGen は，(1) の解析で得た動的解決の記録を登録先の決定に用いる．いずれの決定手順も付録 A.4 に示す．

5. CAPEv2 で観測できる API の調査

インストール直後の CAPEv2 が，回避型マルウェアが呼び出す API をどの程度記録できるかを調査するため，文献 [4] で公開している回避 PoC コード 1,520 件を調査した．

5.1 調査方法

各ソースコードの構造を C/C++コンパイラのフロントエンドである Clang[5] を用いて解析し，抽象構文木上の関数呼び出しノードのうち，Windows SDK のヘッダに宣言を持つものを収集する．ただし，宣言に解決できた呼び出しだけを数えるため，コメントや文字列リテラル中の記述は混入しない．また，GetProcAddress による動的解決についても，引数の文字列リテラルがヘッダ由来の関数名と一致する場合に限り，呼び出しとして数える．抽出した集合には C 標準ライブラリの宣言も含まれるため，コンパイラ組込み関数と C 標準ライブラリの関数を除き，Windows API のみを残すこととする．

続いて，各ソースコードをコンパイルして実行ファイルを CAPEv2 上で実行する．ここで，capemon がフックを設置する API は，hooks.c の登録テーブルに列挙されており，既定の解析で用いられるのは full_hooks テーブルであるため，ここに登録された 586 種を基準として抽出した API と突き合わせる．ただし，フックが設置されていても，LOQ マクロを呼ばない実装では解析レポートに記録が現れないため，これらは観測できないものとして扱う．

5.2 調査結果

はじめに，各ソースコードを構文解析したところ，合計 545 種の API を呼び出すことが確認できた．また，このうち，14 種は Windows API ではなく，__cpuid などのコンパイラ組込み関数 5 種，memcpy や strncpy_s などの C 標準ライブラリおよびその Microsoft 拡張 8 種，C++の演算子 1 種であった．そこで，これらを除いた Windows API 531 種について CAPEv2 での API フックの有無を調査したところ，424 種（79.9%）は観測できなかった．また，このうち，full_hooks に登録がないものは 419 種，登録されていても LOQ を呼ばないものは 5 種であった．このため，API フックが存在することと解析レポートに記録が現れることは同じではなく，観測性を論じるうえではログ出力処理の有無まで見る必要があると言える．

6. 評価実験

本章では提案手法の評価を行う．以降では，4.1 節 (1) の初期状態の capemon による解析をデフォルト，4.1 節の (5) の解析をフック追加後と呼ぶ．

6.1 評価用データセット

評価には，文献 [4] のデータセットに含まれる 1,520 件の回避 PoC から選定した 163 検体を用いる．ただし，1,520 件すべてを処理するには解析とビルドに 300 時間以上を要するため，以下の基準に従って選定した．データセットに含まれるソースコードのうち，サンドボックス検知ツールである Pafish[6] および Al-Khaser[7] を元とする解析回避コード 64 個（データセット A），Galloro ら [2] が体系化した回避技術の分類に基づく解析回避コード 99 個（データセット B）をコンパイルして回避 PoC を作成した．

作成した 163 検体のうち，デフォルトの解析で終了マーカーに到達しなかった検体は，検体もしくは CAPEv2 の事項に問題があると考え，評価の対象から除いた．ただし，同じ検体を同じ構成で解析しても結果が変わることがあるため，到達しなかった検体は解析し直した．21 検体はこれにより完走し，評価の対象に含めている．以上により，評価対象はデータセット A から作成した 63 検体，データセット B から作成した 73 検体の計 136 検体となった．

本実験では，検体の行う挙動を API コールログとしてどれだけ観測できたかを評価する．いずれの検体群もソースコードを伴うため，各検体が明示的に呼び出す Windows API を静的に抽出し，正解集合とした．以降，これをソース明示 API と呼ぶ．なお，4.1 節 (2) でシステムが用いる解析レポート由来の情報は，ソースコードに現れない API を含む一方で，インポートテーブルに載らない動的解決を取りこぼすため，正解集合には用いない．ソース明示 API は，検体ごとに重複を除いて種類を数え，全検体で合計して 549 件である（データセット A：236 件，データセット

© 2026 Information Processing Society of Japan

B：313 件）．同一の検体が同じ API を何度呼び出しても 1 件と数え，異なる検体が呼び出した場合は別に数える．549 件を構成するのは 131 検体であり，残る 5 検体はソース明示 API を使用しないため評価から除いた（6.4 節）．

6.2 実験 1：回避挙動の捕捉率

実験内容：データセット A，B を提案手法を用いて解析した．ただし，データセット A で得たフック DB はデータセット B へ引き継いでいる．CAPEv2 による 1 検体あたりの解析時間は，デフォルトとフック追加後のいずれも 100 秒とした．本実験では，ソース明示 API 549 件のうち，実行時に記録されうるものを評価の対象とする．これは，正しくフックを生成しても，実行されなかった分岐にある API は解析レポートに現れない．ソースコード上のすべての API をレポートと突き合わせると，未実行の分岐にある API まで捕捉の失敗として数えることになるためである．

実行の有無は，検体自身が出力するログを根拠として判定する．検体は処理の進行を `NtWriteFile` を介して出力しており，この API は capemon が既定でフックする．そのため，判定の対象とする API のフックの有無とは独立に，検体がどこまで実行されたかを追うことができる．得られた実行経路をソースコード上の呼び出し箇所と対応づけ，実行された分岐にある API を評価の対象とした．たとえば失敗時にのみ出力されるログが現れていなければ，対応する分岐は実行されておらず，その分岐にある API は対象から外す．関数ポインタ経由の呼び出しやループの成立条件など，ログだけでは実行経路を判定できない例もあるため，549 件すべてについて個別に確認した．

実験結果：実験 1 の結果を**表 3** に示す．ソース明示 API 549 件のうち，デフォルトとフック追加後の双方で呼び出しを確認できたものが 443 件，フックを生成できず除外した API のうち呼び出しを確認できたものが 18 件あり，合わせて 461 件を評価の対象とする．除外した API を評価の対象に含めるのは，その大半が API の性質ではなく本システムがフックを生成できなかったことによるものであり，対象から外せば捕捉率が実力以上に高く出るためである．残る 88 件は呼び出しを確認できなかったものであり，記録すべき対象が存在しないため対象から外した．

以上から，捕捉率は 31.2%から 89.2%へ向上した．記録されなかった 50 件のうち 18 件はフックを作れなかった除外対象の API であり，残る 32 件はフックを追加しても捕捉できなかったものである．データセット別の差は 5.4 ポイントにとどまっており，成り立ちの異なる検体群に対しても同水準の改善が得られている．

フックを追加することによって，補足率は大幅に向上したが，逆にデフォルトでフック実装されていた API について，他のフックを追加することで記録されなくなった事例が 1 件あった．1 件は `UnhookWindowsHookEx` である．

表 3 ソース明示 API の捕捉率（解析レポートに記録された件数を評価対象の件数で除した値）．

	A	B	合計
検体数	63	73	136
評価対象の API（件）	204	257	461
デフォルトで記録（件）	60	84	144
捕捉率	29.4%	32.7%	31.2%
フック追加後に記録（件）	188	223	411
捕捉率	92.2%	86.8%	89.2%

capemon はフックの内側から呼ばれた API をログに記録しない．本手法が追加したフックにより，デフォルトでは記録されていたこの API がフックの内側からの呼び出しと判定されるようになり，記録が失われた．フックを追加することで，かえって観測できなくなる場合がある．

本実験を通じて，システムは 163 検体から 306 種の未フック API を生成対象として特定し，このうち 222 種についてフックを生成して capemon へ組み込んだ．段階ごとの内訳は付録 A.6 に示す．

なお，両データセットを通じて，フックが原因で検体が停止した事例は 0 件であり，停止の原因となったフックを二分探索により特定する手順（A.8 節）は起動しなかった．

また，フック生成の工程に要する時間が 1 検体あたり平均 0.5 秒となり，処理時間の大半は CAPEv2 による解析と capemon のビルドが占め，本手法が加える負荷は小さい．内訳は付録 A.7 に示す．

6.3 実験 2：フック生成規則の妥当性

実験内容：実験 2 では，生成したフックが引数の値を正しく記録するかを検証する．対象は，2 つのデータセットの処理を通じてフック DB に登録された 222 種類のフックすべてである．検証には，データセット A・B の検体とは別に，この目的のために作成したプログラムを用いた．

このプログラムは 222 種類の API を順に呼び出し，各呼び出しの直後に自身が渡した引数の値を `OutputDebugStringA` で出力する．capemon はこの API を既定でフックするため，生成したフックが記録した値と，プログラムが出力した実際の値が同一の解析レポート上に並ぶ．同名の API はプログラム自身の実行時処理からも多数呼ばれるため，各呼び出しを開始と終了のマーカーで挟み，その区間に現れたログのみを照合の対象とした．

実験結果：実験 2 の結果を**表 4** に示す．222 フックのうち 195 フックが解析レポートに API コールを記録することに成功し，記録された引数と実測値を照合できた．照合できた 659 引数のうち 640 引数（97.1%）で値が一致した．型別の内訳は表 4 のとおりで，バッファの内容についてもバイト単位で一致している．照合の対象外とした 85 引数は，プログラム側が比較できる値を出力していないか，フックがアドレスのみを記録しているものである．一致しなかった 27 引数は，型ではなく原因によって 3 つ

© 2026 Information Processing Society of Japan

表 4 生成したフックが解析レポートに記録した内容の照合結果.

検証したフック 222 件（ログあり 195，ログなし 27）
記録された引数 744 個（照合対象 667，対象外 77）

引数の型	照合対象	一致	一致率
整数	232	223	96.1%
戻り値	196	195	99.5%
ポインタ・ハンドル	151	149	98.7%
文字列	53	46	86.8%
パス	17	17	100.0%
バッファの内容	18	10	55.6%
計	667	640	96.0%

に分かれる．最も多い 12 引数は，検証プログラムが持つ期待値が旧実装を前提としていたために生じた見かけ上の不一致であり，記録された値はむしろ正しい．たとえば `RegGetValueA` のデータ引数は，検証プログラムがアドレスを出力しているのに対し，フックはレジストリ値の内容である `Windows 10 Home` を記録していた．アドレスではなく内容が記録されるようになったことの実測証拠である．バッファの一致率が 55.6%と低いのは，不一致 8 個がすべてこの型に該当するためであり，フックの欠陥ではない．規則の欠陥に起因するものは 10 引数であり，ロケールおよび日時を扱う 6 件のフックに集中している．いずれもログ上の表示名を引数名から生成する際に，接頭辞を取り除いた後の一意性を検査していないために生じたものである．`lpLocaleName` と `cchLocaleName` がともに `LocaleName` という同じ表示名に潰れ，後続の整数が文字列を上書きしていた．指定子の選択自体は正しく，表示名の生成に限定された不具合である．残る 5 引数は，同名 API の取り違えや末尾の改行の欠落など，照合手法の限界によるものである．ログが取得できなかった 27 件については付録 A.10 に示す．

6.4 提案手法および評価の限界

提案手法には次の限界がある．第一に，デフォルトで終了マーカーに到達しなかった 27 検体を評価から除いており，これらに対する有効性は確認できていない．第二に，シグネチャ DB に収録されていない API は雛形を生成できず，2 つのデータセットを合わせて 42 種が該当した．第三に，本評価の正解集合はソースコードに現れる API 名を単位とするため，5 検体は件数を持たない．第二と第三の詳細は付録 A.9 に示す．第四に，捕捉の判定は著者らの手作業による．全件に根拠となるログ行を対応づけているが，判定者と実装者が同一であることは避けられていない．

7. まとめ

本研究では，マルウェア動的解析システムの観測性能を高めるため，API フックの生成，解析システムへの組み込み，およびログ取得の検証を自動化する手法を提案した．提案手法は，API のシグネチャからログ出力処理を組み立てる規則を LLM により導出し，これに基づいて生成したフックを組み込んで，ログが取得できることを検証するまでを一連の処理として自動化する．調査により，解析回避の実証コード 1,520 件が呼び出す Windows API 531 種のうち，424 種（79.9%）はデフォルトの CAPEv2 では観測できないことを示した．そのうえで 163 検体を処理して 222 種の API フックを生成し，デフォルトで完走した 136 検体を評価したところ，ソースコードが明示的に呼び出す API のうち解析レポートに記録される割合は 31.2%から 89.2%へ向上した．また，生成したフックが記録した引数の値を実測と照合したところ，照合できた 667 引数のうち 96.0%が一致し，生成規則が高い精度で機能することを確認した．

謝辞 本研究の一部は，国立研究開発法人新エネルギー・産業技術総合開発機構（NEDO）の委託事業「経済安全保障重要技術育成プログラム／先進的サイバー防御機能・分析能力強化」（JPNP24003）によるものである．

参考文献

[1] Kevin O'Reilly. CAPEv2: Malware configuration and payload extraction. `https://github.com/kevoreilly/CAPEv2`. (参照 2026–07–31).

[2] Nicola Galloro, Mario Polino, Michele Carminati, Andrea Continella, and Stefano Zanero. A systematical and longitudinal study of evasive behaviors in Windows malware. *Computers & Security*, Vol. 113, p. 102550, 2022.

[3] Razvan Raducu, Ricardo J. Rodríguez, and Pedro Álvarez. CAPE Hook Generator. `https://github.com/reverseame/cape-hook-generator`, 2024. Version 1.0. (参照 2026–07–31).

[4] 松澤輝, 東志拓, グエンチ ヴァンアン, 田辺瑠偉, インミンパパ, 吉岡克成. サンドボックス解析を回避する実証コード群の LLM を用いた生成. 電子情報通信学会技術研究報告, Vol. 125, No. 381, pp. 224–230, 3 2026. ICSS2025-116.

[5] The LLVM Project. Clang: a C language family frontend for LLVM. `https://clang.llvm.org/`. (参照 2026–08–20).

[6] Alberto Ortega. Pafish: Paranoid fish. `https://github.com/a0rtega/pafish`. (参照 2026–08–15).

[7] Ayoub Faouzi. al-khaser. `https://github.com/LordNoteworthy/al-khaser`. (参照 2026–08–15).

[8] 伊藤曜, グエンティヴァンアン, インミンパパ, 田辺瑠偉, 吉岡克成. LLM によるサンドボックスの回避耐性強化支援. コンピュータセキュリティシンポジウム 2025 論文集, pp. 204–211. 情報処理学会, 10 2025. `https://cir.nii.ac.jp/crid/1050026271562046208`.

[9] Razvan Raducu, Alain Villagrasa-Labrador, Ricardo J. Rodríguez, and Pedro Álvarez. MALVADA: A framework for generating datasets of malware execution traces. *SoftwareX*, Vol. 30, p. 102082, 2025.

[10] Kevin O'Reilly. capemon: CAPE's monitor. `https://github.com/kevoreilly/capemon`. (参照 2026–07–31).

© 2026 Information Processing Society of Japan

8. 研究倫理

本研究の評価には，松澤ら [4] が構築した解析回避の実証コードを用いた．これらは実際のマルウェアではなく，回避技術を再現することを目的として作成されたコードである．同データセットは不適切な利用を避けるために限定的に共有されているものであり，本研究は所定の手続きに従って提供を受けた．本論文においても，実証コードの内容は開示しない．

実証コードの実行は，外部のネットワークから隔離した仮想環境上で行った．また，解析のたびに仮想マシンを実行前の状態へ復元しており，実行の影響が他の解析や環境に及ばないようにしている．

本論文は，デフォルトの CAPEv2 がフックしていない Windows API が多数存在することを示しており，この情報は解析の回避に利用されうる．一方で capemon はオープンソースとして公開されており，フックの対象は誰でも確認できる．本研究はその不足を埋める手法を示すものであることから，公開による不利益は限定的であると判断した．

付　　録

A.1 LOQ の書式指定子

3.3 節で述べた書式指定子のうち，capemon が実装しているものを記録内容によって分類し，**表 A·1** に示す．

表 A·1 LOQ の書式指定子（記録内容による分類）

指定子	記録される内容
`s`,`u`,`S`,`U`	文字列
`f`,`F`	絶対パスへ正規化した文字列
`b`,`B`	バッファの中身
`i`,`I`,`x`,`X`	整数（値渡し／ポインタ経由／ 64bit）
`p`,`P`	アドレス
`o`,`O`,`n`	構造体を展開した文字列・値
`e`,`k`,`v`,`r`	レジストリのパス・値

このうち `I`，`P`，`X`，`O`，`o`，`n` については，capemon 側が例外処理で囲んでポインタの指す先を読む．そのためフック側で参照を行う必要がなく，未初期化のポインタによる異常終了を避けられる．4.2 節の第二の方針は，この性質に依拠している．

A.2 生バッファと長さ引数の対応付け

表 2 の規則 6 について述べる．`PVOID` のような型からは，その引数がバッファであることは分かるが，長さがどこにあるかは型に現れない．そこで，バッファ引数の直後の 2 引数以内に，値渡しの整数であり，かつ名前が `length`，`size`，`count`，`cb`，`cch` などに一致する引数があるかを探索する．該当する引数が存在し，さらに SAL 注釈が入力を示す場合に限り，その引数を長さとして `b` で中身を記録する．

出力バッファでは長さ引数が確保した容量を表し，実際に書き込まれた長さとは一致しないため，中身を記録すると未初期化領域を読むことになる．この場合は `p` に留める．

A.3 規則で決まらない引数の内訳

4.2 節で述べた，規則を適用してもアドレスのみの記録に留まる引数は，次の 3 種に分類できる．

(1) 出力バッファ：書き込まれた長さが戻り値で返されるもの．長さが戻り値であるという事実はシグネチャに現れず，API の仕様を解釈しなければ判断できない．

(2) サイズ埋め込み構造体：`GlobalMemoryStatusEx` の `dwLength` のように，長さが構造体のメンバに格納されているもの．どのメンバが長さかは型ごとの知識を要する．

(3) 任意ポインタ：`CreateThread` の `lpParameter` のように，呼び出し側が自由に定めるコンテキストであり，中身を読む意味がないもの．生バッファと同じ `LPVOID` 型を取るため，型では区別できない．

A.4 カテゴリと登録先 DLL の決定手順

4.3 節で述べた 2 つの決定について，手順を示す．

カテゴリを決めることは，3 節で述べたとおりフック本体を記述するソースファイルを決めることであり，同じ値が `LOQ_`行の `CATEGORY` にも入る．LoggingAPIHookGen はシグネチャ DB が持つカテゴリ表記を第一の判定材料とし，それが得られない場合に限り API 名から推定する．API 名の接頭辞は実際のカテゴリと一致しないことがあるため，推定は補助的な手段に留めている．

登録先 DLL については，シグネチャ DB も API ごとに DLL 名を持つが，実際に検体が経由する DLL と一致するとは限らない．そこで動的解決の記録を用いるほか，相互にフォワードしうる DLL の組をあらかじめ定義し，フック本体は 1 つのまま登録行のみを複数の DLL へ展開する．capemon は当該 DLL がその API をエクスポートしない場合に登録を安全に読み飛ばすため，過剰な登録は無害である．なお，`EnumPrintersW` を提供する `winspool.drv` のように拡張子が `.dll` でないモジュールがあり，これを `.dll` として扱うと解決に失敗するため，DLL 名は拡張子を含めて扱う．

A.5 除外対象 API の選定

4.1 節 (2) の除外対象 API は，当初は LLM が推論により定めた 143 種であり，実験的な証拠に基づいていなかった．そこで評価に先立ち，143 種を対象に個別にフックを生成して単独で capemon へ組み込み，ビルドと解析を行うことで妥当性を検証した．判定は，そのフックを追加したときにログが途切れるかどうかを基準とし，最後まで取得できるものは除外しないこととした．

© 2026 Information Processing Society of Japan

除外を維持したのは 24 種である．第一に，フックを追加すると有害であることを実証できた 2 種である．`RtlCaptureContext` は検体が起動せず，`InitializeSListHead` は実行が停止する．いずれも独立した 2 回の試行で再現している．第二に，本システムが生成またはビルドできない 14 種である．第三に，capemon が実装を持ちながら登録行を欠く 8 種である．これらは同名のフックを新たに生成すると定義が重複してビルドに失敗するため，対象から外している．ただし，本体を生成せず登録行のみを追加すれば対応できる．4.1 節 (2) で述べたログ出力を持たないフックへの対処と同じ発想であり，本システムでは未実装である．検証の過程で，当初の 143 種に含まれていなかった 5 種も第三の分類に該当することが判明した．これを加えた 29 種を除外リストとし，本論文の評価を実施した．

生成またはビルドができない 14 種について，その原因は 3 つに帰属する．(i) シグネチャ DB の不備によるものが 7 種で，うち 4 種は API がそもそも収録されておらず雛形を作れないもの，3 種は戻り値の型に呼び出し規約が混入しており正しい宣言にならないものである．(ii)capemon の実装による制約が 3 種で，2 種は capemon が該当するヘッダを取り込んでいないため引数の型を解決できず，1 種は C++の参照型を引数に取るため，C 言語でビルドされる capemon では表現できない．(iii) ビルド環境の制約が 4 種で，いずれも x64 では生成もビルドも成功しているが，x86 と同時にビルドする構成のため，x86 側の失敗によって全体が失敗と記録される．いずれも生成規則そのものの誤りではなく，入力データ，capemon の実装，ビルド環境のいずれかに原因がある．

他の 119 種は除外の根拠を欠いており，除外を解除した．100 種はフックを追加してもログが最後まで取得でき，除外する理由が確認できなかった．19 種は，生成・ビルド・実行はできるが呼び出しを捕捉できなかったものである．`kernel32` が公開する API の実体が `ntdll` 内の関数への転送である場合が 9 種を占め，本システムがエクスポート元の DLL を登録先とするため実際に実行される実装を捕捉できない．

過剰な除外が生じた原因は，当初の一覧が基準の異なるものを混ぜていたことにある．11 クラスのうち 2 クラスは，フックすると壊れるからではなく，CRT 内部の処理であって行動解析上の価値が低いという理由で挙げられていた．フックを追加すること自体が有害だと実証できたのは 2 種にとどまる．過剰な除外はフックを作らないという形で現れるため，実測しない限り観測性を下げていることに気づけない．除外と生成不能を区別せずに扱うことも，除外の規模を実力以上に大きく見せ，同時に限界を見えなくする．

A.6　フック生成の段階別 API 数

表 A·2 に，本システムが段階ごとに扱った API の種類数を示す．本節の数値は，評価の対象から除いた検体を含む実施 163 検体全体のものである．種類数は検体数に依存するため，データセット間の比較には用いない．和集合は両データセットを通じた種類数であり，両者の和とは一致しない．

検体が使用する API のうち，デフォルトで観測できるのは 4 分の 1 程度にすぎない．延べ件数で見ると，データセット A では 7,834 件のうち 5,808 件（74.1%），データセット B では 11,702 件のうち 8,589 件（73.4%）が未フックであった．デフォルトの capemon は 644 種のフックを備えるが，実際に検体が踏むのは 50〜77 種にとどまる．また，除外対象の 29 種のうち，実際に検体が使用したのはデータセット A で 9 種，データセット B で 7 種であった．

生成の対象となった未フック API のうち，新規の生成を要した割合は延べ件数でデータセット A が 15.4%，データセット B が 11.7%であり，残りはフック DB からの復元で対応できた．データセット B で復元のみで済んだ API が 102 種にのぼるのに対し，空の状態から開始したデータセット A では 1 種にとどまる点に，フック DB を引き継いだ効果が現れている．99 検体を処理しても DB の登録数が 186 件から 222 件へ 36 件増えたにとどまることも，同じ性質を示している．

生成対象の API は和集合で 306 種であり，データセット間で 126 種（41%）が共通する．検体群の成り立ちが異なっても，回避のために踏まれる API には重なりが大きく，これがフック DB の再利用が効く理由である．

生成できなかった API は，いずれもシグネチャ DB に収録されていないものであった．種類で見るとデータセット A で 31 種，データセット B で 32 種であり，両者の和集合は 42 種，共通は 21 種と顔ぶれがほぼ一致する．新しく追加された kernel32 の API，非公開の ntdll 関数，ws2_32 の内部エントリ，標準 C ライブラリやロケール処理の内部関数が該当する．

表 A·2　データセット A・B のフック生成の段階別 API 種類数．

段階	A	B	和集合
検体が使用する API	280	295	400
デフォルトでフック済み	50	77	85
除外 API	9	7	9
生成対象（未フック）	221	211	306
シグネチャ未収録で生成不能	31	32	42
既存フックへログ出力を追加	3	2	3
フック DB から復元	1	102	—
新規に生成	186	75	—
検証を通りフック DB へ登録	186	136	222

© 2026 Information Processing Society of Japan

A.7　処理に要した時間

表 A·3 に，各段に要した時間を示す．本節の数値は，評価の対象から除いた検体を含む実施 163 検体全体のものである．本システムが実際に計算を行うフック生成の工程は，全体の 0.06%，1 検体あたり平均 0.5 秒にすぎない．生成対象のうち新規の生成を要したのは延べ件数で 11.7〜15.4%であり，残りはフック DB からの復元で済むためである（付録 A.6）．残りの時間は CAPEv2 による解析（約 77〜79%）と capemon のビルド（約 21〜23%）である．1 検体あたりの所要時間は中央値で約 10 分であった．

ビルドの回数は 1 検体あたり 1.08〜1.33 回であり，1 検体につき 1 回のビルドで済むことが基本である．超過分はビルド失敗の再試行と，可読性の修正後の再ビルドによる．解析の回数が 1 検体あたり 2.6〜2.7 回となるのは，初期状態と追加後の 2 回が基本構成であるためである．

表 A·3　各段に要した時間

段	データセット A	データセット B	占有率
フック生成	0.5 分	0.8 分	0.06%
ビルド	157.8 分	283.8 分	約 21〜23%
解析	604.8 分	976.1 分	約 77〜79%
累計	12.7 時間	21.0 時間	

A.8　停止の原因となるフックの特定

4.1 節 (5) で終了マーカーを観測できず，かつ (1) では到達していた場合に限り，原因となるフックを二分探索により特定する．追加したフックの集合を 2 分し，一方のみを組み込んだ capemon をビルドして再解析することを繰り返すことで，停止を引き起こす API を絞り込む．探索そのものはスクリプトが決定的に行い，LLM は (1) との照合に基づいて探索を実施するか否かを判断する．特定した API は修正するか，修正できない場合は除外対象へ加える．

A.9　評価の限界の補足

6.4 節の第二の限界について述べる．cape-hook-generator は，シグネチャ DB に該当がない場合に API 名を Web 検索して Microsoft Learn のページを取得し，SAL 注釈つきのプロトタイプと登録先 DLL を抽出する機能を備えている．本研究ではこの機能を用いていないが，外部の検索サービスとページ構造への依存を許容できるのであれば，該当する 42 種の一部は生成できると考えられる．

第三の限界について述べる．ソース明示 API を持たない 5 検体は，性質の異なる 2 種に分かれる．3 検体は，CPU 命令である `__cpuid` や `KUSER_SHARED_DATA` の固定アドレスからの直接読み出しによって環境を判定しており，関数呼び出しを伴わないため API フックでは原理的に捕捉できない．残る 2 検体は `_popen` や `std::filesystem` を経由しており，これらは内部で Windows API を呼ぶため挙動そのものは観測できている．ソースコード上の名前と解析レポート上の名前が一致しないために正解集合へ載らないだけであり，提案手法ではなく評価手法の限界である．

A.10　ログが得られなかったフック

6.3 節の検証において，222 件のうち 27 件はログが 1 件も得られなかった．このうち 3 件は，プロセスの制御を壊すなどの理由で検証プログラムから意図的に呼び出していない．`ExitProcess` など 3 件は，本手法がログ出力を元の API の呼び出しの後に置く設計上，戻らない関数では到達しないことによる．

残りは，呼び出したにもかかわらずログが得られなかったものである．同期プリミティブ，ヒープ操作，プロセスやスレッドの情報取得，高分解能タイマが大半を占める．これらは付録 A.5 で述べた，`kernel32` の API が実体として `ntdll` へ転送される場合に該当するものが多く，実検体を用いた検証とは異なる経路でありながら同じ結論に至った．

© 2026 Information Processing Society of Japan